

FLASH Development Guide

Last updated October 23, 2025.

Written by Daniel Terry and Manuel Juette.

1 TABLE OF CONTENTS

2	Introduction	2
3	Prerequisites	2
4	Structure	3
4.1	Subsystems	3
4.2	Class Hierarchy	4
4.3	Configuration files	5
5	Walkthrough	5
5.1	General program flow	5
6	Creating a new actor class	8
6.1	Creating a new <i>Device</i> actor	8
6.2	Creating a new <i>UI Window</i> actor	9
6.3	Creating a new method	10
6.4	Adding and accessing class properties	10
6.5	Creating a message	10
6.6	Editing a message	11
6.7	Global variables	11
6.8	Actor Core.vi	11
6.9	Asynchronous Communication	12
7	Adding support for new devices	13
8	Other notes and guidelines	14
8.1	Compiling an executable	14
8.2	BinaryTIFF	14
8.3	Team development	15

2 INTRODUCTION

FLASH is an instrument control software package for widefield single-molecule fluorescence measurements, especially smFRET. This document provides an overview of the code base and tips for adding new features. Before making any changes, please review the [LabVIEW programming paradigms](#).

The architecture is based on the LabVIEW [Actor Framework](#) and aims to provide enough abstraction and modularity to allow for relatively easy adaptation to different setups. This guide explains specific design features and development workflows for FLASH. It is not a general introduction to the Actor Framework or LabVIEW. A good starting point to learn the basics of the actor framework is the *Actor Framework Fundamentals* project from the Example Finder, along with documents found at <https://goo.gl/A86KuP>.

Please read “FLASH Documentation.pdf” first before this document to review the normal functioning of the program.

3 PREREQUISITES

Prerequisites for running the compiled FLASH executable are listed in “FLASH Documentation.pdf”. For development, the following *additional* requirements need to be met:

All libraries supported by FLASH must be installed to avoid link errors, even if the developer doesn’t plan to use them. It is possible to bypass this restriction by removing a device llib from the project and ensuring that it is never directly referenced in the source code.

- 64-bit LabVIEW Professional version 2023 Q3 (newer versions may also work but are untested). This requires a license to be purchased. <https://www.ni.com/en-us/shop/labview.html>
 - JSONtext from JDP Science (download using VI Package Manager).
- Install latest version of Hamamatsu Video Capture library for LabVIEW. This will install VIs into LabVIEW user.lib folder and install tmcamcon.dll to C:\Windows\System32. <https://dcam-api.com/hamamatsu-software/>
- Andor SDK2 (for EMCCD cameras): tested with version 2.104.30084.0. <https://andor.oxinst.com/downloads/uploads/Software%20Development%20Kit.pdf>
<https://andor.oxinst.com/downloads/view/andor-sdk-2.104.30084.0>
- Photometrics PVCAM and LabVIEW adapter. FLASH depends on the current BETA version of the LabVIEW adapter (2.3.x), which is not publicly available. Please directly contact Photometrics to request the latest version. <https://www.photometrics.com/products/pvcam>
<https://www.photometrics.com/support/third-party-software/labview>
- Nikon Ti2 SDK 64-bit, which includes the device driver for the Ti2 microscope. <https://nisdsk.recollective.com/microscopes>
- Thorlabs Kinesis SDK for LabVIEW. https://www.thorlabs.com/software_pages/ViewSoftwarePage.cfm?Code=Motion_Control

- ASI stage controller driver for LabVIEW.
<https://www.asiimaging.com/support/downloads/ms-2000-control-using-nis-labview-and-the-serial-port/>
- Coherent Genesis driver. Your laser should have included a CD that will install an application called “OPSL” and the driver. If you need to install the driver directly, it can be found in a directory similar to “CDM v2.12.28 WHQL Certified” on the CD.
<https://www.coherent.com/support>
- Optional: Visual Studio with C/C++ tool chain (free Community Edition is sufficient) to recompile the C DLLs (specifically `dcimg2tiff.dll`) used for time-critical operations in FLASH.
- Optional: instructions for installing software for configuring the Arduino are list on the developer’s website: <https://github.com/stjude-smc/microsync>

Once these packages are properly installed, deploy the FLASH source code to “C:\FLASH\Development”. To run FLASH, open the project file, double click on “FLASH.vi” from within the LabVIEW project, and click the “play” button to start the program. Do not start “FLASH.vi” from outside the project!

4 STRUCTURE

4.1 SUBSYSTEMS

FLASH is divided into the following four subsystems:

1. **User Interface (UI) layer.** Includes *Main Window* and *Viewer* actors that together handle all user interactions. These Actors communicate via messages only to the *Application* Actor and do not interact directly with the hardware layers. Ideally, the UI does not maintain state – it mirrors the state of the Application, as indicated by status messages.
2. **Application layer.** This Actor is the core system that controls the lifecycle of all the other Actors and defines the functional implementation of the actions requested via the UI layer. The Application maintains the state of the program and orchestrates multi-step workflows.
3. **Hardware abstraction layer (HAL).** Translates device agnostic commands (like “start streaming”) into device-specific commands (DLL calls or RS232 signals). Each class in this layer encapsulates the functionality of a device type (*Camera*, *Microscope*, etc.). The public methods of the class are called by the application layer via a message, describing a high-level command. The public method then calls protected methods in the same class that abstract device-specific commands (such as “prepare camera”). These protected methods are overridden by device-specific classes so that dynamic dispatch can be used to call the correct device-specific class. HAL classes do not maintain state.
4. **Device layer.** Provides actors for all supported hardware devices. The methods override those in the HAL to include device-specific API calls. In some cases, the HAL and Device layers are merged, with specific devices handled by Case Structures within each method. This approach minimizes development time but is only appropriate for simple devices (e.g., *Power Meter*).

4.2 CLASS HIERARCHY

The main classes within FLASH are actors, which are derived from the Actor Frameworks *Actor* class. The core *Application* actor implements the Application subsystem. *UI Window* and *Device* are abstract classes that provide generic functionality for the UI and Hardware subsystems; specific windows and devices are implemented by subclassing. Messages between actors are derived from a separate *Message* class, which is also part of the framework.

Actor (LabVIEW built in type)

- Application
- TIFF Writer (Service)
- Particle Counter (Service)
- UI Window
 - Main Window
 - Viewer
 - Splash Screen
- Device
 - Camera
 - 1. Hamamatsu
 - 2. Andor (SDK2)
 - 3. Photometrics
 - Laser
 - 1. LaserQuantum
 - 2. Coherent (Obis and HOPS families)
 - 3. Thorlabs (polarizer motion control)
 - 4. Cobolt
 - Microscope
 - 1. Nikon Ti2
 - Power Meter
 - 1. Newport
 - 2. Thorlabs
 - Stage Controller
 - 1. Ti2
 - 2. ESP30x
 - 3. Ludl
 - 4. ASI
 - Sync Device
 - 1. NI DAQ
 - 2. Arduino DAQ ([Mega2560](#); legacy support)
 - 3. Microsync

AutomationContext (encapsulates automation state)

4.3 CONFIGURATION FILES

FLASH stores instrument configuration settings in files in the “Config” folder. Version 1.5.3 and later use the JSON format for compact, human-readable text that is robust to changes in the type definitions between versions. Previous versions used an XML format based on LabVIEW’s own schema and strictly-typed clusters, which was extremely sensitive to even the smallest changes in type definitions. Both formats can be read, but only JSON is supported moving forward.

The *Init.json* file in the main project directory contains general settings, including a map of computer host names and default configuration files to use for each. This mechanism aids in deploying the software to facilities with a large number of microscopes. Currently this file can only be modified in a text editor; there is no user interface for it in FLASH.

5 WALKTHROUGH

Always open the project file (.lvproj) in the FLASH directory to examine the code or make any changes. Avoid running or modifying files outside the project.

The main subsystems are contained in virtual folders, including *Application*, *Hardware*, and *User Interface* described above. The *Resources* virtual folder contains type defs, global variables, dependencies, and various VIs used for development/testing.

Within the virtual folders for the different subsystems, each actor is contained in a separate LabVIEW library (.lvlib) file. This includes:

1. A virtual folder with the message classes for this actor.
2. The actor class itself. This, in turn, contains the private data cluster and separate virtual folders for protected and public methods, getters and setters. Notes:
 - i. LabVIEW does not enforce any particular organization of class members. Adhering to a fixed virtual folder structure prevents confusion.
 - ii. Each public method corresponds to a message class and vice versa, as this is the framework’s mechanism to expose functionality to calling actors.

At the top level, there is only one VI (*FLASH.vi*), which is a minimal launcher for the top-level *Application* actor.

5.1 GENERAL PROGRAM FLOW

This section describes the general execution flow for a typical workflow. The names of the messages listed here generally have a corresponding message handler vi with the same name in the receiving actor’s source code. Except for the *Splash Screen* actor, the lifespan of all actors is the same as the application.

- 1) *FLASH.vi* launches the *Application* root Actor.
- 2) *Application/Actor Core.vi* launches the *Splash Screen* Actor.

- 3) *Application/Load Configuration.vi* dialog allows user to select the hardware configuration, saving this information in the *Application* class data. Modifying configuration uses “Create Hardware Configuration UI.vi” dialog.
- 4) *Application/Init Hardware.vi* launches device actors, as dictated by the selected configuration file. The *Splash Screen* updates as each device completes its initialization procedure.
- 5) Once the hardware is ready, *Splash Screen* is closed, and the *Main Window* actor is launched.
- 6) *Main Window/Actor Core.vi* contains an Event Structure, which translates user interactions to Messages sent to the *Application* actor. For example, clicking “Stage Forward” sends a “Stage Forward” message to *Application*, which is then forwarded to the *Stage* object.
- 7) Some device actors send regular status update messages to the *Application* that are forwarded to *Main Window/Actor Core.vi* and immediately update the appearance of the user interface elements. For example, after manual stage repositioning, the *Stage* actor will send the current stage position to be displayed as a red dot in the *Main Window*.
- 8) The state of *Main Window* controls is dictated by the state of the *Application*. This process controlled by “Application Mode Event” messages, which in term enable/disable controls according to what actions are possible during the current application mode.
- 9) In response to clicking the “Show Live” button:
 - a. *Main Window* UI controls describing acquisition settings are validated, packaged into a “Camera Settings” cluster object, and sent to the *Application* with a “Start Live” message.
 - b. *Application/Start Live.vi* performs additional checks (maximum shutter rate, etc.).
 - c. Application mode is changed from “Idle” to “Live Mode” and an “Application Mode Event” message is sent to *Main Window* to disable front panel controls.
 - d. A synchronous “Start Live” message is sent to the *Camera* actor to prepare for acquisition using camera settings passed from *Main Window*. Completion of this step is communicated via the “acquisition ready” global variable.
 - e. Once the cameras are ready, a “Start Live” is sent to the synchronization device (if configured) to open shutters and trigger camera exposures.
 - f. The *Viewer* actor is launched (or made visible again), displaying live frame data from the cameras via the “Live Frame” global variable.
 - g. With each new frame, *Camera/Get Most Recent Frame (if active).vi* saves the raw frame data into the “Live Frame” global variable and sends a “New Frame Notify” message to the *Application* actor, which is also forwarded to the *Viewer* actor.
 - h. When user clicks “Stop Live” in *Main Window* or closes the *Viewer* window, a “Stop Live” message is sent to the *Application* actor. This sends a “Stop Live” message to the *Camera* and *Sync Device* actors, and the *Viewer* actor is sent a message that hides the

window (but does not stop the actor). Finally, an “Application Mode Event” message (Idle state) is sent to the *Main Window* to re-enable front panel controls.

10) In response to clicking the “Stream Acquisition” button:

- a. *Main Window* actor validates and collects acquisition settings into a “Movie Series” cluster. This is then sent to the *Application* in a “Start Recording” message.
- b. *Application/Start Recording.vi* does some additional checks, including validity of the filename, enough disk space, etc. A “Start Recording Event” User Event is then sent to the helper loop in *Application/Actor Core.vi*, which coordinates the timing of the automation steps below through the entire movie series.
- c. Optional: the *Stage* actor is sent a message to move forward to the next field.
- d. Optional: execute autofocus routine (see next section).
- e. *Application/Actor Core.vi* sends a “Start Stream” message to the *Application* actor (itself).
- f. Application mode is changed from “Idle” to “Streaming” and an “Application Mode Event” message is sent to *Main Window* to disable front panel controls.
- g. A synchronous “Start Streaming” message is sent to the *Camera* actor to prepare for acquisition using Camera Settings cluster passed from *Main Window*. Completion of this step is communicated via the “acquisition ready” global variable.
- h. Once the cameras are ready, a “Start Streaming” is sent to the synchronization device (if configured) to open shutters and trigger camera exposures.
- i. The *Viewer* actor is launched (or made visible again), displaying live frame data from the cameras via the “Live Frame” global variable.
- j. With each new frame, *Camera/Get Most Recent Frame (if active).vi* saves the raw frame data into the “Live Frame” global variable and sends a “New Frame Notify” message to the *Application* actor, which is also forwarded to the *Viewer* actor. Simultaneously, the camera driver saves frame data to an intermediate file on disk (.raw or .dcimg).
- k. When the *Application* actor receives a “New Frame Notify” message that the last frame has been acquired, it executes “Finish Stream.vi”. This then sends “Finish Streaming” messages to *Sync Device* and *Cameras*, resets the application state to “Idle”, and closes the *Viewer* window.
- l. *Camera/Finish Stream.vi* stops the acquisition and sends a message to the *TiffWriter* actor to start a job translating the raw frame data saved to intermediate files on disk (.raw or .dcimg) into the final .tif file. This process runs asynchronously, allowing the next acquisition to proceed. *Main Window* displays the saving process with a progressbar at the bottom of the window.

- m. This process repeats from the beginning until the full movie series has been acquired. In each cycle, the helper loop in *Application/Actor Core.vi* waits for notifications of the application state changing to “Idle” before proceeding to the next step.

11) Autofocus: to be completed.

12) When the Main Window is closed by the user, it sends a stop signal to the *Application* actor. No specific shutdown code is required within the *Application* actor, as all nested actors are set to “auto-stop” together with their calling actor (at which point they all execute their respective termination code). If an actor does not respond to the shutdown message (such as if a message handler method got stuck in an infinite loop), the program may continue executing the background and must be stopped with Task Manager.

6 CREATING A NEW ACTOR CLASS

6.1 CREATING A NEW *DEVICE* ACTOR

Below is the general process for creating a new *Device* actor.

Inheriting from an existing sub-class of *Device* (e.g., *Camera*, *Laser*, *Microscope*):

This is the most common scenario.

- Create a new class inheriting from the *Device* class, or a more specific class that defines a specific device type such as *Camera*, *Laser*, or *Sync Device*.
- For later compilation, be sure to add this class to the “always included” list under the “source files” tab of the project build specification.
- Add any device-specific data to the properties in the .ctl file associated with the class.
- Create a public “Initialize.vi” method that takes an uninitialized instance (default value) of the class and uses its inputs to set class properties and prepare for execution. Any startup tasks that take time (such as connecting to a device) should be done in this vi. This VI is synchronous, so execution will stall until it completes.
- Add a value for the new device in the “Device Type” enum in the hardware configuration cluster for that device type.
- Add code to launch the new actor type to “Application/Init HW.vi” or the nodes therein. In many cases, this requires adding a case with the path to the .lvclass file so the actor can be dynamically launched. Use the enum value created in the previous step.
- Create an override for “Shut Down Device” for any steps to cleanly shut down the actor, including shutting down the device and closing any device handles.
- Create override methods for any other methods as needed. For example, classes inheriting from *Camera* will override “Start Live.vi”, “Stop Live.vi”, and “Get Most Recent Frame (if active).vi” for viewing frame data.

- Many override methods should call the parent method to set variables, trigger events that are common to all devices of this type, and handle errors. Look at other classes inheriting from the same base class to determine when this should be done.
- For devices from derived classes such as *Camera*, *Laser*, *Microscope*, and *Sync Device*, dynamic dispatch will be used to direct a message defined in the parent class to the specific class method of the actual device class. As such, these classes do not need to define any messages of their own.
- To create a new method that can be triggered by a message that is not an override of an existing method, you must create a new Message class. Right-click on the public method and choose Actor Framework->Create Message. Then use the new "Sendvi" node to send the new message.

Inheriting directly from *Device* class (used for entirely new type of device):

- Optional: create a new cluster typedef for the configuration of the new device. This will typically include any connection settings (like COM port) and an enum listing device sub-types (such as vendor or product name). Include this new typedef in the Hardware Configuration typedef so it will be included in configuration files.
- Create an abstract base class inheriting from *Device* with methods that define the interface other actors should use. Look at other derived Actor classes for inspiration on interface design. Create messages for each public method. There is no need to override all methods of *Device* or *Actor*. Be sure to create an "Initialize.vi" that takes the above configuration typedef as input.
- Create a new concrete class implementing the interface defined by the above abstract class.
- Add code to launch the new Actor from "Application/Init HW.vi". Best practice is to load the actor from file path using the "Get LV Class Default Value" node so that the module is only loaded when the device is actually installed. The "Initialize.vi" node should be called before finally launching the nested actor.

Note about using the *Pre Launch Init* method: actors CANNOT be launched there (attempting this will cause the program to hang without any error message). *Pre Launch Init* should only be used to check if pre-requisites for running the core are met (e.g. parameters initialized?) and abort launching the actor if necessary.

6.2 CREATING A NEW *UI WINDOW* ACTOR

User interface windows that need to run continuously and asynchronously from other actors should be instances of the *UI Window* class. For many modal dialog type use cases, it is best to use a simple VI wired directly into the calling VI with the front panel visible. The VI will block execution until the task is complete. For an example of this use case, see "Countdown Timer.vi".

Here are the basic steps for creating a new *UI Window* class:

- Create a new class inheriting from *UI Window*
- Create an override of the “Create Front Panel Events” that calls “Create User Event.vi” for each type of message that may need to be passed to Actor Core. Bundle these into the class data and update the class .ctl file to include them.
- Create an override for “Destroy Front Panel Events” that calls “Destroy User Event.vi” to clean up all of the User Event instances.
- Create an override method for Actor Core.vi. This will generally include all user interface elements on the front panel and a helper loop with an Event Structure in the wire diagram. User interactions with front panel controls trigger cases in the Event Structure and perform the desired actions. Be sure to wire the various User Event properties from the class data into the Event Structure so Actor Core can receive these events.
- Alternatively, references to front panel controls can be added to the class members that are updated via properties nodes in message handler methods. This approach is useful for handling device status updates where no action is needed other than updating a front panel control (such as the progress bar in the *Viewer* window).

IMPORTANT: Don’t wire TRUE to “Show Front Panel” option when launching the actor, as this will not work for built applications. Use VI properties of Actor Core.vi file instead.

6.3 CREATING A NEW METHOD

First decide whether the method should be public (often used for message handling) or private (internal methods with no associated messages). To override the behavior of a parent class, right click on the virtual folder associated with this scope and select “Add->VI for Override”. To add a method that is specific to the class, right-click on the fold and select “Add->VI from dynamic dispatch template”.

Utility methods that do not use class data should be created as an ordinary VI and added to the .lvlib outside of the class hierarchy.

6.4 ADDING AND ACCESSING CLASS PROPERTIES

Class properties are defined in a .ctl file in each class, to which new properties can be directly added. These are easily accessed by class methods using the cluster “bundle/unbundle by name” Vis wired from the class instance wire. To access properties inherited from the base class (such as *Device* or *Camera*), either use property nodes or create a new method in the parent class specifically designed for this task.

6.5 CREATING A MESSAGE

Actors generally communicate by asynchronous message passing. This approach minimizes coupling between actors and allows them to run simultaneously and independently with minimal chances of race conditions.

Messages are processed by the actor in a first-in-first-out order (not in parallel). As such, any method used to respond to a *Message* should generally execute in minimal time; tasks that may block while waiting for a resource to become available or are computationally intensive should not be executed

directly in message handling methods. These are best processed within Actor Core.vi (or passed along to some other actor specialized for this task).

To create a new *Message* for an existing method, first make sure the target vi is in the “Public” scope and its connector pane has been finalized, including which inputs are required, as this will be harder to change later. Next, right click on the desired method vi and select “Actor Framework->Create Message”. This will execute a script to create a new message class, which should reside in the “Messages for this actor” virtual folder. The class properties include all elements for each of the VI inputs. The methods include a Do.vi that calls the target method and a “Send <method name>.vi” that is used to send a message to the target actor. None of these files should be edited directly by the developer.

It is possible to send messages synchronously by inheriting from the “Send Message And Wait For Response” VI, but this is not recommended as it can create race conditions and was intentionally made by NI to be difficult to implement.

6.6 EDITING A MESSAGE

Changes to class methods do not generally require any changes to the associated message class. However, if you have changed the connector pane of a method VI, including which terminals are required, you need to update the associated message class. This is accomplished by right-clicking on the message class and choosing “Actor Framework->Rescript message”.

6.7 GLOBAL VARIABLES

Whenever feasible, all communication between actors should be done by passing messages, but there are some exceptions.

For performance and memory usage reasons, it is not recommended to pass large blocks of data in a *Message* (such as image data). Instead, use a global variable in “Resources/GLOBAL Time-Critical.vi” or pass a Data Value Reference (DVR). Extra care must be taken in these instances to ensure that the data flow is unidirectional with a single writer, and the data is protected from race conditions.

Some status variables are communicated synchronously, such as “Trigger Ready” and “Current Frame”. These are stored “Resources/GLOBAL Time-Critical.vi” to bypass the overhead associated with message passing and simplify the implementation. This should be avoided when possible and is likely to change in the future.

Another global variable (“GLOBAL Experiment Metadata.vi”) is used to store device status and acquisition settings to be used for saving metadata to movie files. This is done to simplify the implementation to avoid creating a large number of new messages but is likely to be removed in future versions.

6.8 ACTOR CORE.VI

Actor Core.vi runs continuously throughout the lifetime of the actor. This VI typically splits the actor instance wire to the parent Actore Core, which implements a “message handler” loop, and a [“helper loop”](#) (in the override method). The helper loop runs in its own thread separate from message handling, which creates several issues that the developer should keep in mind:

- The ‘stale’ actor object wire is not updated when message handling methods modify the class data over time. This wire is still useful, for example for referencing the device configuration, which cannot be changed in FLASH after initialization.
- Class methods should not be called directly from Actor Core using the stale actor object wire. Instead, use “Read Caller Enqueuer.vi” to self-enqueue messages, which will call class methods using the active reference to the class data.
- Class methods can communicate with the Actor Core helper loop in a few ways:
 - a. **User Events:** “Create User Event” VI creates a reference to a queue that is saved in class properties. Class methods call the “Generate User Event” VI to add an event to the queue. Actor Core wires a reference to the queue to its Event Structure and add cases to react each user event type. The class destructor should call “Destroy User Event” to clean up these references. This approach is ideal for reactive actors such as *UI Window* instances.
 - b. **Data Value Reference (DVR):** a DVR can be referenced by both class methods and Actor Core to refer to the same memory location. This approach is most valuable for time-critical applications such as in *Synchronization Device*. The flow of information should be unidirectional and extra care must be taken to avoid race conditions.
 - c. **Global variables:** this approach should be avoided if possible. It is currently used in FLASH for broadcasting some information across the whole application, such as the current frame number. This behavior will be changed in a future version.
 - d. Notifiers, queues, and some other objects are passed by reference and therefore can be used for communication between Actor Core and other class methods, but this isn’t standard in FLASH. This can include references to front panel controls, which can be useful for real-time updates of the user interface elements.

6.9 ASYNCHRONOUS COMMUNICATION

One of the most challenging aspects of the Actor Framework for many developers is the asynchronous nature of the message passing architecture. Rather than executing an imperative sequence of steps in series, Actors send messages to other Actors that execute tasks asynchronously. This works well when no reply is needed and steps can be executed simultaneously.

One solution to provide synchronous execution is for the sending actor to poll a variable (with a global variable, functional global variable, data value reference, queue etc.) that is modified by the receiving actor when the task is complete. This will block the actor from processing any messages during this time, which could lock the application. As such, this approach is only used in FLASH for short steps such as waiting for cameras to be ready for acquiring data. The process also includes a timeout to prevent the application from locking completely.

Another solution is for the sending actor to be configured as a state machine with member variable being checked by every message handler vi to determine how to react to various messages it receives. The next step in a process can then be executed when the actor receives a reply that the requested task

has been completed by the other actor. This is generally the preferred method, and is used by the Application actor, but is more complex to implement.

7 ADDING SUPPORT FOR NEW DEVICES

This section describes in more detail the specific steps needed for adding support for a new device. Refer to the sections above to learn how to create a new Actor class. Generally speaking, each vendor will require a new class, but specific models from that vendor can be implemented as distinct cases within the methods of that class, unless the implementation uses a completely different library, as is the case with Andor EMCCD (SDK2) vs sCMOS (SDK3) cameras.

Power meter: support for new power meters can be added by adding a case to each of the files in “Hardware\Power Meter” and updating the associated typedef and config files. This class does not use dynamic dispatch.

Stage: to add support for a new stage, modify files in “Hardware\Stage Controller\”. Each of these has a Case Structure for each of the supported devices. Be aware that some stages use the same API as the microscope stand, so there may be overlap (see below). This class does not use dynamic dispatch.

Microscope: to add support for a new microscope stand, a new class inheriting from the *Microscope* class is needed. In cases where the stage is controlled by the microscope, the microscope instance should open and close the connection to the device and the handle should be shared with the Stage instance. This can be implemented with a functional global variable.

Sync Device:

- Many National Instruments DAQ devices are likely supported by simply modifying the NI-DAQmx physical channel addresses in the configuration file.
- Other Arduino-type devices can be supported if they can be programmed to accept the expected commands.
- To add an entirely new device type, create a new class inheriting from *Sync Device*.
- Be sure that “Initialize.vi” for your class outputs the correct “Jitter (ms)” value that reflects the absolute worst case variance in trigger timing. Do not set this to zero – this is a special value that indicates the absence of an installed synchronization device.

Shutters: any shutter driver that can be triggered by TTL (0/+5V) logic should be supported. Other optical modulators, such as AOM, EOM, and AOTF devices, can be similarly controlled if they can be programmed to accept TTL signals from the DAQ. FLASH does not support controlling shutters directly via USB or RS232 connections because these are too slow to ensure synchronizing with frame acquisition from the cameras.

Lasers: support for controlling laser power directly at the laser controller or via some attenuator device (such as a motorized rotating polarizer) can be added by creating a new class inheriting from *Laser*.

Fluidics: FLASH does not currently support extensive control of solution control devices such as perfusion pumps, but Sync Device can be configured to send a TTL pulse to such a device that is configured to accept it.

Cameras:

- New camera models from vendors support by FLASH (Hamamatsu and Photometrics) may not require any source code changes, but if they do, first look at Prepare Acquisition.vi as this includes the code that is most likely to be device specific.
- To support a new vendor, create a new actor modeled after the Hamamatsu actor, inherited from the Camera class (see previous section).
- **BinaryTIFF**: for new vendors, you may also need to add a new function to the dcimg2tiff library and new case to “Merge DCIMG Files.vi” to call it. This will be used to transform the raw data saved to disk by the camera driver into a tiff file.

8 OTHER NOTES AND GUIDELINES

8.1 COMPILING AN EXECUTABLE

Except for testing and development, FLASH should always be run as a compiled binary. To compile the executable, follow these steps. Version numbers are specified as MAJOR.MINOR.BUGFIX.BUILD. Major versions break significant functionality users may depend on (such as migrating the new file formats). Minor versions implement significant new features. Bugfixes are slight changes to the code that only correct a specific bug and add no new features. The build number is used internally for development purposes and generally corresponds to each commit in the source code repository.

- 1) In the project, double-click on “Build Specifications->FLASH”.
- 2) Click on the “Version Information” tab and update the version number. Build number should be set to be automatically incremented with each build.
- 3) If new files are required that LabVIEW may not find by traversing the VI dependencies (such as image and dll files or classes referenced only by file path), these can be added in the “Source Files” page. This also includes any classes that are dynamically loaded at runtime via their file path, which is used for many device actors.
- 4) Click “Build” to compile the project.
- 5) The destination for the compiled project should be “C:\FLASH\testing” for unstable versions to be tested and “C:\FLASH\master” for stable releases deployed to production workstations.
- 6) Most DLLs should be saved in the root directory with FLASH.exe. Some libraries will be automatically saved by LabVIEW and others need to be copied there manually. Some libraries are referenced by path; these are generally stored in the “dll” folder.

8.2 BINARYTIFF

This is a standalone LabVIEW/C++ project used to combine raw files streamed from the camera library to disk into TIFF files, where the different spectral channels are tiled side-by-side (up to four channels in a 2x2 grid), after any image flipping if requested. We use this approach to allow the device driver to stream directly to disk to avoid dropped frames. For Hamamatsu cameras, the utility reads .dcimg files using the DCIMGAPI library. For Photometrics, raw pixel data are saved that can be read directly. For further documentation, see the project directory.

8.3 TEAM DEVELOPMENT AND OTHER TIPS

Due to LabVIEW's graphical nature and binary source code, version control has some unique challenges but is doable. Below are some tips to make the process as painless as possible:

- 1) When moving files, always do so in LabView using the "Move on disk" option in the Files view – do not move files outside of LabView!
- 2) It is best to use a single shared computer for producing compiled versions. This ensures the build number is always correct. This can be implemented as a nightly script.
- 3) Only commit files that were intentionally changed to limit scope. LabVIEW has a tendency to alter files that the user did not touch for obscure reasons. For example, great care should be taken in committing the main project file (".lvproj") to avoid conflicts. This should not generally be necessary unless a new module has been added. In general, the project file should only be committed with a major change.
- 4) Never commit the ".aliases" and ".lvls" files, which are workstation specific. Use .gitignore if possible to exclude them explicitly.
- 5) Although possible, merging multiple commits to a single file isn't recommended. If multiple contributors are working on the code base at the same time, take extra effort to minimize the scope of changes to avoid possible conflicts. If larger scale changes are needed (such as to widely-used typedefs), communication is essential to coordinate commits without conflicts.
- 6) It is possible to diff or merge commits using LVDIFF and LVMERGE utilities, but requires some work to set up and is finicky. https://labviewwiki.org/wiki/Set_up_differencing_capabilities
- 7) Changes to typedefs (including enums) will cascade through every file that uses the typedef, which increases the likelihood of conflicts. Avoid changes to typedefs when possible. Typedefs are only needed when it is desirable for every piece of code using that typedef to need review if the data structure is modified. When used, keep the scope of any typedefs confined only to where it is needed.
- 8) Review this article about separating source and compiled code. If this option is not selected, source files are more likely to be altered indirectly and cause issues for version control. <https://www.ni.com/docs/en-US/bundle/labview/page/facilitating-source-control-by-separating-compiled-code-from-vis-and-other-file-types.html>